

数字时钟和自动报时系统设计

一、课程设计目的

- 1.了解以微机为核心的接口方式访问接口设备的原理，提高接口扩展硬件电路的连接能力。
- 2.了解数字时钟和语音报时的工作原理，提高应用系统的设计与调试的综合能力
- 3.加深对定时器/配件、配件接口芯片和语音芯片的硬件电路及软件编程的工作方法和编程方法的理解。

二、课程设计内容

设计数字时钟，能够动态时间显示，整点语音报时。利用定时器产生精确的时钟源，通过更换或者查询的方式将时钟源数值发送给微机控制台，微机控制台在进行数据的处理后，控制控件接口进行 DLED 的动态扫描，正确的显示时钟；利用语音芯片 ISD1420 进行编程录音，记录整点报时音。系统运行时之后，数码管显示时分秒，每到整点，喇叭播放报时音。

三、系统功能与设计要求

1. 系统功能与设计要求：

系统显示界面：

- 在 DLED 数码管上进行实时刷新显示。
- 使用 8255 扫描数码管，显示出时分秒，小时、分钟分别为显示，秒使用 LED3 的 DP 显示，0.5 秒亮，0.5 秒灭。

自动报文时要求：

- 通过 Smashing 实现语音录制、播放，可以随时修改报时音内容。
- 报时音以 24 时制录音，24 个报时音以连续地址存放在 ISD1420 芯片内，录音棚顺序如下如表所示。

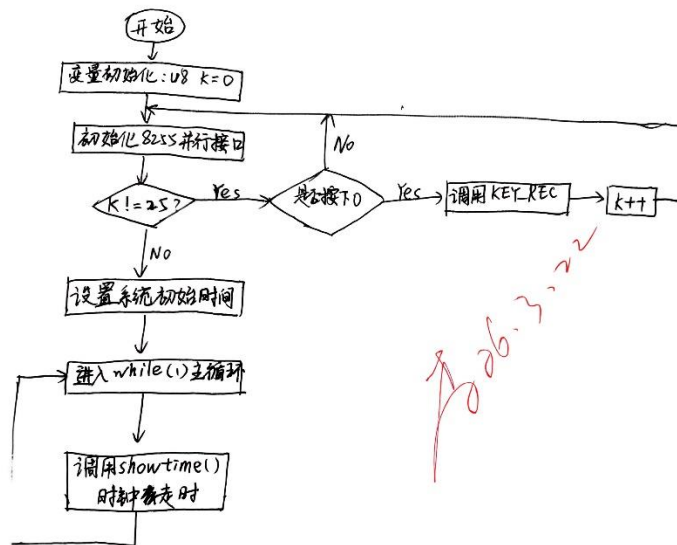
表 5-3-1 报时录音存放地址表

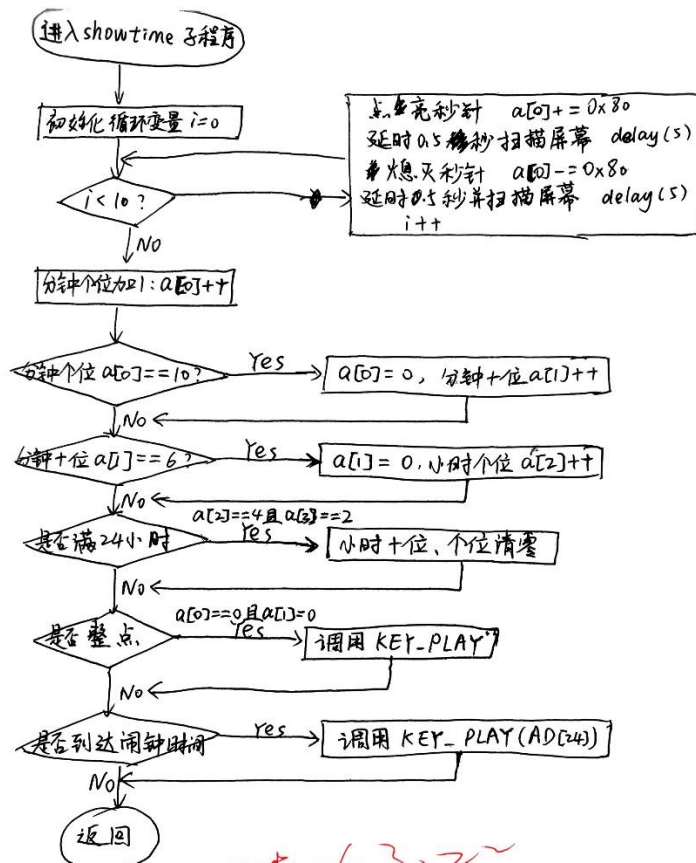
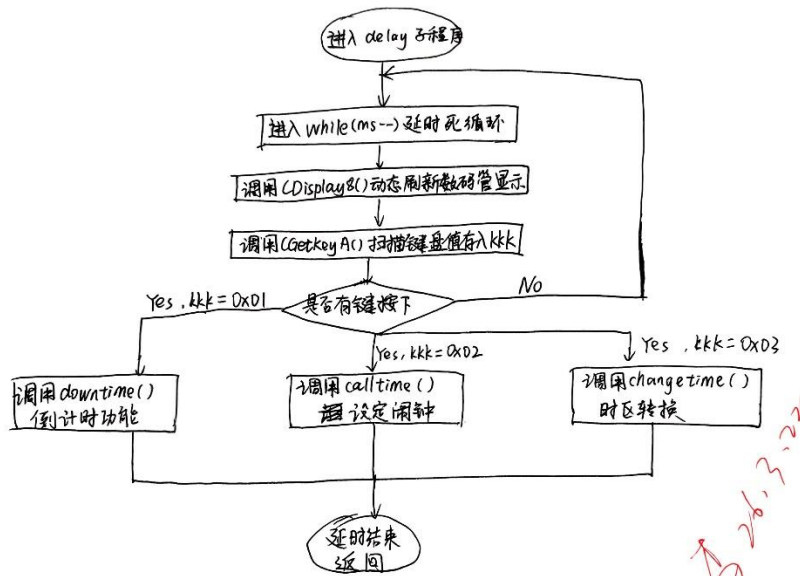
地址	报时音	地址	报时音	地址	报时音	地址	报时音
00	零点整	06	六点整	12	十二点整	18	十八点整
01	一点整	07	七点整	13	十三点整	19	十九点整
02	两点整	08	八点整	14	十四点整	20	二十点整
03	三点整	09	九点整	15	十五点整	21	二十一点整
04	四点整	10	十点整	16	十六点整	22	二十二点整
05	五点整	11	十一点整	17	十七点整	23	二十三点整

2. 发挥部分

- 增加“语音提醒”功能，自行设计提醒时间和播放的语音内容。打开“语音提醒”功能后，数字钟运行到
- 设定的时刻，喇叭会发出声音提醒预先录制的语音。
- 增加“表秒计时”功能，启动该功能后可以在 DLED 上进行 00.00~99.99 秒计时。
- 增加“时区转换功”功能，根据设定的时区表随时转换为相应时区的时刻。

四、软件流程图





五、实验过程

1. 硬件连接准备

1. 8255 芯片连接：

- 配置 PA 端口(0x270)为输出模式，用于控制 ISD 语音模块的地址
- 配置 PB 端口(0x271)用途未显式显示(可能在外部函数中使用)
- 配置 PC 端口(0x272)的低四位为输出，高四位为输入，用于控制 ISD 语音模块的模式

2. ISD 语音模块连接：

- 连接到 8255 的 PA 和 PC 端口
- PA 端口提供录音/放音的起始地址
- PC 端口控制 REC、PLAYE、PLAYL 等控制信号

3.显示和键盘接口：

- 使用外部函数 cDisplay8() 和 cGetKeyA() / cGetKeyB() 控制显示和读取按键

2. 初始化过程

1. 8255 初始化：

- 通过写入控制寄存器(0x273)值 0x81，设置 PA 输出、PB 输出、PC 低四位输出和高四位输入

2. ISD 语音模块初始化：

- 调用 initISD()函数，设置 PC 端口为 0x70(关闭所有录放音功能)
- 设置 PA 端口为 0，允许手动录放音

3.主要功能测试

3.1 时间显示功能(showtime ())

1.数码管显示：

- 使用仓库 a[] 存储时间数据，格式为[秒个位，秒十位，分个位，分十位, ...]

- 每 60 次循环(约 1 秒)更新一次时间
- 秒个位数字会有引发效果(通过交替加/减 0x80 实现)

2.时间进位逻辑:

- 秒(00-59)、分(00-59)、小时(00-23)的自动进位
- 处理特殊 2 点之后的 4 分(24 小时制)

3.整点报时:

- 当分钟和都秒为 0 时, 播放一小时的语音(AD 内存中对应的地址)

3.2 倒计时功能(downtime ())

1.设置倒计时:

- 通过按键 1 输入倒计时时间(分:秒格式)
- 按 15 键确认输入完成

2.倒计时执行:

- 每秒递减时间
- 当时间归零时播放提示音(AD[24])

3.3 闹钟功能 (通话时间 ())

1. 设定闹钟时间:

- 通过键盘 2 输入闹钟时间
- 按 15 键确认输入完成

2.闹钟触发:

- 在 showtime() 中检查当前时间是否与闹钟时间匹配
- 匹配时间播放提示音(AD[24])

3.4 时区转换(changetime ())

1.转换逻辑:

- 读取当前小时数(存储在 a[3]和 a[2])
- 改变一个时区(增加两小时)

4.语音模块功能测试

1. 录音功能(注释状态):

- 代码中包含录音功能 KEY_REC() ， 但主函数中被注释
- 可以录制 25 段语音(0.5 秒每段)，存储在 AD 备份指定的地址

2. 放音功能:

- KEY_PLAY() 用于播放指定地址的语音的函数
- 整点报时、倒计时结束、闹钟触发时都会调用

六、实验收获与收获

本次实验我主要负责系统设计，可以从数字时钟和自动报时两个部分考虑，数字时钟部分可以利用延迟循环获取大致时间，根据时钟计算出时分秒，然后驱动 4 位数码管实时显示出时间；自动报时主要完成语音录制和播放功能，可以通过配件接口 8255 控制 ISD1420 的 REC\PLAYE\PLAYL 端口，然后编写录音和放音子程序，系统可以通过 8255 硬件接口 PA 口控制录音\播放，PB 口和 PC 口控制时间显着表示数码管的段、位。

本次实验通过基于 8255 和 ISD 语音模块的嵌入式时钟系统开发，深入实践了硬件接口控制、多任务调度与人机交互设计。核心收获包括：掌握了端口配置与语音模块的操作操作，强化了编程通过调试思维解决了语音无响应、时间输入错误等典型问题，并认识到硬件初始化与软件逻辑良好的实验不仅提升了嵌入式系统开发能力，还对未来启示可通过引入实时操作系统(RTOS)、优化低功耗设计、扩展网络同步功能等方式进一步优化系统，为复杂项目积累了宝贵经验。

七、代码展示

```
#define PA_Addr 0x270
#define PB_Addr 0x271
#define PC_Addr 0x272
#define CON_Addr 0x273
#define u8 unsigned char
#define u16 unsigned int

extern char inportb(unsigned int);
extern void outportb(unsigned int, char);
extern void cDisplay8(u8*);
extern u8 cGetKeyA();
extern u8 cGetKeyB();
extern void cInitKeyDisplay();

void delay(u16 ms, u8* a);

u8 kkk;
u8 AD[25] = {
    0x00, 0x04, 0x08, 0x0C, 0x10,
    0x14, 0x18, 0x1C, 0x20, 0x24,
    0x28, 0x2C, 0x30, 0x34, 0x38,
    0x3C, 0x40, 0x44, 0x48, 0x4C,
    0x50, 0x54, 0x58, 0x5C, 0x60
};

u8 a[] = {0x09, 0x05, 0x03, 0x02, 0x10, 0x10, 0x10, 0x10};
u8 b[] = {0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0x10};
u8 c[] = {0x00, 0x00, 0x00, 0x00, 0x10, 0x10, 0x10, 0x10};

void delay1(u16 ms) {
    u16 i;
    while (ms--) {
        i = 100;
        do {
            ;
        } while (--i);
    }
}

u8 KeepMode;
```

```

void initISD() {
    outportb(PC_Addr, 0x70);
    KeepMode = 0x70;
    outportb(PA_Addr, 0);
}

void Delay50ms() {
    u16 i = 6600;
    while (--i) {
        ;
    }
}

void Delay_025S() {
    u16 i = 16000;
    while (i--) {
        ;
    }
    i = 16000;
    while (i--) {
        ;
    }
}

void stopISD() {
    u8 i = (KeepMode & 0xbf);
    outportb(PC_Addr, i);
    i |= 0x40;
    outportb(PC_Addr, i);
    i |= 0x30;
    outportb(PC_Addr, i);
    KeepMode = i;
    outportb(PA_Addr, 0);
    Delay50ms();
    Delay50ms();
    Delay50ms();
}

void recISD(u8 startadr) {
    u8 i;
    outportb(PA_Addr, startadr);
}

```

```

        i = KeepMode & 0xef;
        outportb(PC_Addr, i);
        KeepMode = i;
    }

void playISD(u8 startadr) {
    u8 i;
    stopISD();
    outportb(PA_Addr, startadr);
    i = KeepMode & 0xdf;
    outportb(PC_Addr, i);
    i |= 0x20;
    outportb(PC_Addr, i);
    KeepMode = i;
}

u8 KEY_REC(u8 startadr) {
    u16 count = 2;
    recISD(startadr);
    while (count--) {
        Delay_025S();
    }
    stopISD();
    return 0xff;
}

u8 KEY_PLAY(u8 startadr) {
    u16 count = 2;
    playISD(startadr);
    while (count--) {
        Delay_025S();
    }
    return 0xff;
}

void downtime() {
    u8 mykey;
    char p = 3;

    b[0] = 0; b[1] = 0; b[2] = 0; b[3] = 0;
    CDisplay8(b);
}

```

```

while (1) {
    mykey = CGetKeyB();
    if (mykey == 15) {
        break;
    }
    if (p >= 0) {
        b[p] = mykey;
        CDisplay8(b);
        p--;
    }
}

while (1) {
    delay(54, b);

    if (b[0] == 0 && b[1] == 0 && b[2] == 0 && b[3] == 0) {
        KEY_PLAY(AD[24]);
        break;
    }

    if (b[0] > 0) {
        b[0]--;
    } else {
        b[0] = 9;
        if (b[1] > 0) {
            b[1]--;
        } else {
            b[1] = 5;
            if (b[2] > 0) {
                b[2]--;
            } else {
                b[2] = 9;
                if (b[3] > 0) {
                    b[3]--;
                }
            }
        }
    }
}

}

void calltime() {

```

```

    u8 mykey;
    u8 p = 3;
    cDisplay8(c);
    while (1) {
        mykey = cGetKeyB();
        if (mykey == 15) {
            break;
        }
        c[p] = mykey;
        cDisplay8(c);
        p--;
    }
}

void changetime() {
    u8 pp = a[3] * 10 + a[2];
    pp += 2;
    pp %= 24;
    a[3] = pp / 10;
    a[2] = pp % 10;
}

void delay(u16 ms, u8* a) {
    while (ms--) {
        cDisplay8(a);
        kkk = cGetKeyA();
        if (kkk != 0xff) {
            if (kkk == 0x01) {
                cDisplay8(b);
                downtime();
            } else if (kkk == 0x02) {
                calltime();
            } else if (kkk == 0x03) {
                changetime();
            }
        }
    }
}

void showtime() {
    u8 i;
    for (i = 0; i < 10; i++) {

```

```

        a[0] += 0x80;
        delay(5, a);
        a[0] -= 0x80;
        delay(5, a);
    }
    a[0]++;
    if (a[0] == 10) {
        a[0] = 0;
        a[1]++;
        if (a[1] == 6) {
            a[1] = 0;
            a[2]++;
        }
        if (a[2] == 10 || a[2] == 4) {
            if (a[2] == 10) {
                a[2] = 0;
                a[3]++;
            } else if (a[2] == 4) {
                if (a[3] == 2) {
                    a[2] = 0;
                    a[3]++;
                }
            }
        }
        if (a[3] == 3) {
            a[3] = 0;
        }
        if (a[0] == 0 && a[1] == 0) {
            KEY_PLAY(AD[a[3] * 10 + a[2]]);
        }
    }
    if (a[0] == c[0] && a[1] == c[1] && a[2] == c[2] && a[3] == c[3]) {
        KEY_PLAY(AD[24]);
    }
}

```

```

void main() {

```

```

    u8 k = 0;
    outportb(CON_Addr, 0x81);

```

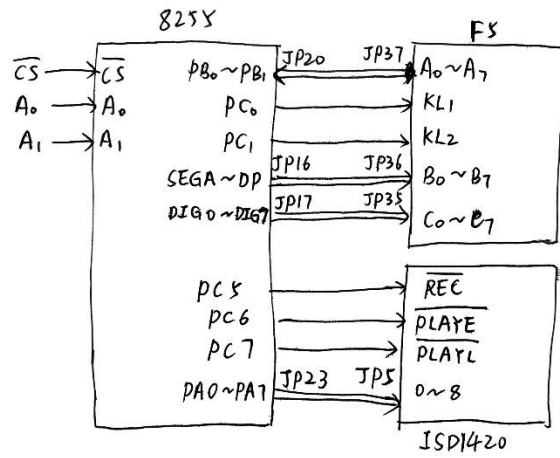
```

    while (k != 25) {

```

```
        if (cGetKeyB() == 0x00) {  
            KEY_REC(AD[k]);  
            k++;  
        }  
    }  
  
    while (1) {  
        showtime();  
    }  
}
```

八、硬件连线图



263.22

